

Manual paso a paso para conectar Supabase a una app móvil Android Kotlin con ViewModel

Fuente base exclusiva: documentación oficial de Supabase para Android Kotlin: <https://supabase.com/docs/guides/getting-started/quickstarts/kotlin>

Objetivo: guiar la conexión de una base de datos de Supabase a una aplicación móvil Android en Kotlin que utiliza `ViewModel`, respetando exclusivamente la lógica y los elementos documentados por Supabase en su quickstart oficial.

Nota de alcance: la documentación oficial hace la consulta desde el código de UI y aclara que en producción probablemente debería usarse un `ViewModel` para separar la UI de la lógica de datos. Esta guía toma exactamente esa observación y reorganiza la llamada para que viva en el `ViewModel`.

1. Idea central de la integración

La conexión puede entenderse como un flujo de cuatro capas:

1. Supabase crea y expone los datos.
2. Android obtiene permiso para conectarse a Internet.
3. La app inicializa un cliente Supabase.
4. El `ViewModel` consulta los datos y la UI únicamente los representa.

2. Crear el proyecto en Supabase

Ingresa a `database.new` y cree un proyecto nuevo de Supabase.

La guía oficial también menciona la posibilidad de crear proyectos mediante la Management API, pero para un flujo práctico directo se parte del panel de Supabase.

3. Crear la tabla y cargar datos de ejemplo

Una vez que el proyecto esté listo, abra el **SQL Editor** y ejecute el siguiente script oficial:

```
-- Create the table
create table instruments (
  id bigint primary key generated always as identity,
  name text not null
);

-- Insert sample data into the table
```

```
insert into instruments (name)
values
('violin'),
('viola'),
('cello');

-- Grant the privileges the role needs, which is read access
grant select on public.instruments to anon;

-- Enable row level security for the table
alter table instruments enable row level security;
```

Qué hace este bloque

- Crea la tabla `instruments`.
- Inserta tres registros de prueba.
- Otorga permisos de lectura al rol `anon`.
- Activa Row Level Security (RLS).

4. Crear la política RLS para lectura pública

Después ejecute la política oficial indicada por Supabase:

```
-- Create a policy to allow the anon role to read from the instruments table
create policy "public can read instruments"
on public.instruments
for select to anon
using (true);
```

Este paso es indispensable porque activar RLS sin una política deja la tabla protegida, pero inaccesible para la lectura desde la aplicación cliente.

5. Exponer la tabla en la Data API

En el Dashboard de Supabase, abra **Integrations > Data API** y exponga las tablas o funciones que quiera acceder desde la aplicación.

Si desea que futuras tablas del esquema `public` queden expuestas automáticamente, puede habilitar **Default privileges for new entities**.

6. Crear el proyecto Android Kotlin

En Android Studio:

1. Seleccione **New**.
2. Seleccione **New Android Project**.

7. Instalar dependencias

Abra el archivo `build.gradle.kts` del módulo `app` y agregue lo indicado por la guía oficial:

```
plugins {  
    // ...  
    kotlin("plugin.serialization") version "$kotlin_version"  
}  
  
dependencies {  
    // ...  
    implementation(platform("io.github.jan-tennert.supabase:bom:$supabase_version"))  
    implementation("io.github.jan-tennert.supabase:postgrest-kt")  
    implementation("io.ktor:ktor-client-android:$ktor_version")  
}
```

Reemplace los marcadores:

- `$kotlin_version`
- `$supabase_version`
- `$ktor_version`

La documentación referencia estos recursos para consultar versiones:

- [supabase-kt releases](#)
- [Ktor docs](#)

Función de cada dependencia

Dependencia	Propósito
<code>kotlin("plugin.serialization")</code>	Permite serializar y deserializar datos entre JSON y objetos Kotlin.
<code>postgrest-kt</code>	Permite consultar tablas expuestas por Supabase mediante PostgREST.
<code>ktor-client-android</code>	Provee el cliente HTTP para Android.

8. Agregar permiso de Internet

En el archivo `AndroidManifest.xml`, dentro de `manifest` y fuera de `application`, agregue:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Sin este permiso, la aplicación no podrá comunicarse con Supabase.

9. Obtener la URL y la key del proyecto

Según la documentación, debe obtener:

- **Project URL**
- **Publishable key**

Estas se obtienen desde el diálogo **Connect** del proyecto.

La guía también explica que Supabase está migrando desde las claves heredadas `anon` y `service_role` hacia nuevas claves, y recomienda usar la **publishable key** en el cliente.

Punto crítico: para una aplicación móvil se debe usar la **publishable key**. La clave de servidor no debe colocarse del lado cliente.

10. Inicializar el cliente de Supabase

La guía oficial muestra la siguiente inicialización:

```
val supabase = createSupabaseClient(  
    supabaseUrl = "https://xyzcompany.supabase.co",  
    supabaseKey = "your_publishable_key"  
) {  
    install(Postgrest)  
}
```

Debe reemplazar `supabaseUrl` y `supabaseKey` con los valores reales de su proyecto.

Aunque la guía ubica esta inicialización cerca de `MainActivity.kt`, en una arquitectura con `ViewModel` puede mantenerse como objeto compartido siempre que conserve exactamente estos elementos:

- `createSupabaseClient`
- `supabaseUrl`
- `supabaseKey`
- `install(Postgrest)`

11. Crear el modelo de datos

La documentación crea una clase serializable llamada `Instrument`:

```
@Serializable  
data class Instrument(  
    val id: Int,
```

```
        val name: String,  
    )
```

Esta clase representa cada fila devuelta desde la tabla `instruments`.

12. Llevar la consulta al ViewModel

La guía oficial realiza la consulta con `LaunchedEffect` dentro del composable, pero explícitamente indica que en producción probablemente conviene usar un `ViewModel` para separar UI y datos.

La adaptación mínima es la siguiente:

```
class InstrumentsViewModel : ViewModel() {  
  
    var instruments by mutableStateOf<List<Instrument>>(emptyList())  
    private set  
  
    fun loadInstruments() {  
        viewModelScope.launch {  
            withContext(Dispatchers.IO) {  
                instruments = supabase  
                    .from("instruments")  
                    .select()  
                    .decodeList<Instrument>()  
            }  
        }  
    }  
}
```

Qué hace el ViewModel

- Conserva el estado de la lista `instruments`.
- Ejecuta la consulta a Supabase.
- Transforma la respuesta en una lista tipada de `Instrument`.

La operación central de Supabase sigue siendo exactamente la del quickstart oficial:

```
supabase.from("instruments").select().decodeList<Instrument>()
```

13. Conectar la UI al ViewModel

La interfaz deja de consultar la base de datos directamente. En lugar de eso, le pide al `ViewModel` que cargue datos y solo se encarga de mostrarlos.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)
```

```

        setContent {
            SupabaseTutorialTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    InstrumentsScreen()
                }
            }
        }
    }
}

```

Composable de pantalla:

```

@Composable
fun InstrumentsScreen(
    viewModel: InstrumentsViewModel = viewModel()
) {
    val instruments = viewModel.instruments

    LaunchedEffect(Unit) {
        viewModel.loadInstruments()
    }

    LazyColumn {
        items(instruments, key = { instrument -> instrument.id })
    }
    { instrument ->
        Text(
            instrument.name,
            modifier = Modifier.padding(8.dp)
        )
    }
}

```

Diferencia arquitectónica respecto al ejemplo de Supabase

En el quickstart	En la versión con ViewModel
La UI consulta datos.	El ViewModel consulta datos.
La UI mantiene el estado.	El ViewModel mantiene el estado.
La UI mezcla representación y acceso a datos.	La UI solo representa el estado recibido.

14. Código mínimo integrado

```
val supabase = createSupabaseClient(
    supabaseUrl = "https://xyzcompany.supabase.co",
    supabaseKey = "your_publishable_key"
) {
    install(Postgrest)
}

@Serializable
data class Instrument(
    val id: Int,
    val name: String,
)

class InstrumentsViewModel : ViewModel() {

    var instruments by mutableStateOf<List<Instrument>>(emptyList())
    private set

    fun loadInstruments() {
        viewModelScope.launch {
            withContext(Dispatchers.IO) {
                instruments = supabase
                    .from("instruments")
                    .select()
                    .decodeList<Instrument>()
            }
        }
    }
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SupabaseTutorialTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    InstrumentsScreen()
                }
            }
        }
    }
}

@Composable
```

```

fun InstrumentsScreen(
    viewModel: InstrumentsViewModel = viewModel()
) {
    val instruments = viewModel.instruments

    LaunchedEffect(Unit) {
        viewModel.loadInstruments()
    }

    LazyColumn {
        items(instruments, key = { instrument -> instrument.id })
    } { instrument ->
        Text(
            instrument.name,
            modifier = Modifier.padding(8.dp)
        )
    }
}

```

15. Ejecutar la aplicación

Ejecute la app en un emulador o dispositivo físico usando **Run app** en Android Studio.

Si todo fue configurado correctamente, deberían mostrarse los registros de ejemplo:

- violin
- viola
- cello

16. Flujo resumido

En Supabase	En Android
1. Crear proyecto. 2. Crear tabla instruments. 3. Insertar datos. 4. Activar RLS. 5. Crear policy de lectura. 6. Exponer la tabla en Data API. 7. Obtener URL y publishable key.	1. Crear proyecto Kotlin. 2. Agregar dependencias. 3. Agregar permiso de Internet. 4. Inicializar cliente Supabase. 5. Crear modelo Instrument. 6. Consultar desde ViewModel. 7. Mostrar datos con Compose.

17. Problemas más probables

1. La tabla existe pero no devuelve datos

- Verifique que ejecutó `grant select on public.instruments to anon;`
- Verifique que activó RLS.

- Verifique que creó la policy de lectura.

2. La app no conecta

- Revise `supabaseUrl`.
- Revise `supabaseKey`.
- Revise el permiso de Internet en `AndroidManifest.xml`.

3. La consulta compila pero falla al decodificar

- Confirme que el modelo Kotlin coincide con los campos de la tabla: `id` y `name`.

4. La app abre pero la lista aparece vacía

- Revise si insertó los datos de prueba.
- Revise si la tabla fue expuesta en **Integrations > Data API**.

18. Conclusión

La guía oficial de Supabase para Android Kotlin enseña un recorrido directo: crear proyecto, preparar la tabla, habilitar acceso, inicializar el cliente y consultar mediante PostgREST. El uso de `ViewModel` no cambia la integración con Supabase; únicamente traslada la responsabilidad de obtención y mantenimiento del estado fuera de la UI para lograr una arquitectura más limpia.

Documento generado a partir de la conversación actual, usando exclusivamente como fuente técnica la guía oficial de Supabase para Android Kotlin.